

EulixOS

面向 RISC-V 的 llama.cpp RVV 算子优化

OSPP 2026 Project Proposal

Applicant: 孔嘉乐

Affiliation: 华中师范大学

Major: 华为“智能基座”基地班

Email: jialekong2004@mails.ccnu.edu.cn

Mentors: 杨旺

2026 年 8 月 25 日

目录

1 项目背景	1
1.1 项目基本要求	1
1.2 项目仓库	1
2 GGML RISC-V 现状	1
2.1 目前实现	1
2.2 SSM_SCAN 的 RVV 缺口	2
2.3 Gated-Delta-Net 缺少算子级 RVV 路径	2
2.4 量化点积没有充分适配 VLEN=1024	3
3 项目实现方案	4
3.1 为 SSM_SCAN 增加 RVV 向量化路径	4
3.1.1 实施目标	4
3.1.2 实现	4
3.2 为 Gated-Delta-Net 增加算子级 RVV 优化路径	5
3.2.1 实施目标	5
3.2.2 实现	5
3.3 面向 VLEN=1024 的量化点积内核优化	6
3.3.1 实施目标	6
3.3.2 实现	7
4 项目计划	7
4.1 第一阶段：代码分析与基线建立（第 1-2 周）	7
4.2 第二阶段：实现 SSM_SCAN 的 RVV 路径（第 3-5 周）	8
4.3 第三阶段：实现 Gated-Delta-Net RVV 优化路径（第 6-8 周）	8
4.4 第四阶段：优化 VLEN=1024 量化点积（第 9-10 周）	9
4.5 第五阶段：综合测试与性能评估（第 11 周）	9
4.6 第六阶段：代码整理与上游协作（第 12 周）	10
5 参考文献	10

1 项目背景

1.1 项目基本要求

OSPP 官网: <https://summer.ospp.ac.cn/org/prodetail/26a190058>

1.2 项目仓库

项目成果仓库: <https://gitee.com/eulixos/llama.cpp>

llama.cpp 主仓库: <https://github.com/ggml-org/llama.cpp>

2 GGML RISC-V 现状

2.1 目前实现

```
GGML Computation Graph
|
v
GGML CPU Backend
|
+--> Common CPU operators
|
+--> RVV SIMD abstraction
|   |
|   +--> vec.cpp / vec.h
|   +--> arch/riscv/quants.c
|   +--> arch/riscv/repack.cpp
|   +--> llamafile/sgemm.cpp
|
+--> SpacemiT backend
|   |
|   +--> spacemit/rvv_kernels.cpp
|   +--> spacemit/ime.cpp
|   +--> IME GEMM / repack / norm / attention
```

2.2 SSM_SCAN 的 RVV 缺口

```

9707 #if defined(GGML_SIMD)
9708     #if defined(__ARM_FEATURE_SVE)
9709         const int ggml_f32_epr = svcntw();
9710         const int ggml_f32_step = 1 * ggml_f32_epr;
9711
9712         const int np = (nc & ~(ggml_f32_step - 1));
9713
9714         GGML_F32_VEC sum = GGML_F32_VEC_ZERO;
9715
9716         GGML_F32_VEC adA = GGML_F32_VEC_SET1(dA);
9717         GGML_F32_VEC axdt = GGML_F32_VEC_SET1(x_dt);
9718
9719         for (int i = 0; i < np; i += ggml_f32_step) {
9720             // Implement with Codex
9721             // TODO: maybe unroll more?
9722             for (int j = 0; j < 1; j++) {
9723                 GGML_F32_VEC t0 = GGML_F32_VEC_LOAD(s0 + i + j*ggml_f32_epr + ii*nc);
9724                 GGML_F32_VEC t1 = GGML_F32_VEC_LOAD(B + i + j*ggml_f32_epr + g*nc);
9725                 GGML_F32_VEC t2 = GGML_F32_VEC_LOAD(C + i + j*ggml_f32_epr + g*nc);
9726
9727                 t0 = GGML_F32_VEC_MUL(t0, adA);
9728                 t1 = GGML_F32_VEC_MUL(t1, axdt);
9729
9730                 t0 = GGML_F32_VEC_ADD(t0, t1);
9731
9732                 sum = GGML_F32_VEC_FMA(sum, t0, t2);
9733
9734                 GGML_F32_VEC_STORE(s + i + j*ggml_f32_epr + ii*nc, t0);
9735             }
9736
9737             sumf = GGML_F32_VEC_REDUCE_ONE(sum);
9738         }
9739     #elif defined(__riscv_v_intrinsic)
9740         // Implement with Codex
9741         // TODO: RVV implementation

```

图 1: SSM_SCAN 的 RVV 主循环尚未实现，后续 state 更新仍走标量路径。

2.3 Gated-Delta-Net 缺少算子级 RVV 路径

```

10859 if (kda) {
10860     // precompute exp(g) into delta scratch (reused below)
10861     for (int64_t i = 0; i < S_v; ++i) {
10862         delta[i] = expf(g_d[i]);
10863     }
10864     // S[i][:] *= exp(g[i]) => for each row j of M: M[j][i] *= exp(g[i])
10865     for (int64_t j = 0; j < S_v; ++j) {
10866         ggml_vec_mul_f32(S_v, &s_out[j * S_v], &s_out[j * S_v], delta);
10867     }
10868 } else {
10869     ggml_vec_scale_f32(S_v * S_v, s_out, expf(g_d[0]));
10870 }
10871
10872 // delta[j] = sum_i S[i][j] * k[i] = dot(row j of M, k)
10873 for (int64_t j = 0; j < S_v; ++j) {
10874     float sum = 0.0f;
10875     ggml_vec_dot_f32(S_v, &sum, 0, &s_out[j * S_v], 0, k_d, 0, 1);
10876     delta[j] = (v_d[j] - sum) * beta_val;
10877 }
10878
10879 // outer product: S[i][j] += k[i] * delta[j] => M[j][i] += delta[j] * k[i]
10880 for (int64_t j = 0; j < S_v; ++j) {
10881     ggml_vec_mad_f32(S_v, &s_out[j * S_v], k_d, delta[j]);
10882 }
10883
10884 // attn_out[j] = sum_i S[i][j] * q[i] = dot(row j of M, q)
10885 for (int64_t j = 0; j < S_v; ++j) {
10886     float sum = 0.0f;
10887     ggml_vec_dot_f32(S_v, &sum, 0, &s_out[j * S_v], 0, q_d, 0, 1);
10888     attn_data[j] = sum * scale;
10889 }

```

图 2: 状态矩阵衰减、点积、外积更新和输出计算仍由多层标量循环组织

```

1142 // TODO For GGML_OP_GATED_DELTA_NET
1143 // case GGML_OP_GATED_DELTA_NET:
1144 //     return true;

```

图 3: 底层向量 helper 存在，但算子级 RVV 和硬件路由未完成

2.4 量化点积没有充分适配 VLEN=1024

```
void ggml_vec_dot_q4_0_q8_0(int n, float * GGML_RESTRICT s, size_t bs, const void * GGML_RESTRICT vx, size_t bx, const void * GGML_RESTRICT vy, size_t by, int nrc) {
    #if defined(__riscv_v)
        const int qk = QK8_0;
        const int nb = n / qk;

        assert(n % qk == 0);
        assert(nrc == 1);
        UNUSED(nrc);
        UNUSED(bx);
        UNUSED(by);
        UNUSED(bs);

        const block_q4_0 * GGML_RESTRICT x = vx;
        const block_q8_0 * GGML_RESTRICT y = vy;

        int lb = 0;
        float sumf = 0;

        size_t vl = qk / 2;

        for (; lb < nb; ++lb) {
            // load elements
            vuint8m1_t tx = __riscv_vle8_v_u8m1(x[lb].qs, vl);

            vint8m1_t y0 = __riscv_vle8_v_i8m1(y[lb].qs, vl);
            vint8m1_t y1 = __riscv_vle8_v_i8m1(y[lb].qs+16, vl);

            // mask and store lower part of x, and then upper part
            vuint8m1_t x_a = __riscv_vand_vx_u8m1(tx, 0x0F, vl);
            vuint8m1_t x_l = __riscv_vsrl_vx_u8m1(tx, 0x04, vl);

            vint8m1_t x_ai = __riscv_vreinterpret_v_u8m1_i8m1(x_a);
            vint8m1_t x_li = __riscv_vreinterpret_v_u8m1_i8m1(x_l);

            // subtract offset
            vint8m1_t v0 = __riscv_vsub_vx_i8m1(x_ai, 8, vl);
            vint8m1_t v1 = __riscv_vsub_vx_i8m1(x_li, 8, vl);

            vint16m2_t vec_mul1 = __riscv_vmmul_vv_i16m2(v0, y0, vl);
            vint16m2_t vec_mul2 = __riscv_vmmacc_vv_i16m2(vec_mul1, v1, y1, vl);

            vint32m1_t vec_zero = __riscv_vmv_v_x_i32m1(0, vl);
            vint32m1_t vs2 = __riscv_vwredsum_vs_i16m2_i32m1(vec_mul2, vec_zero, vl);

            int sumi = __riscv_vmv_v_s_i32m1_i32(vs2);

            sumf += sumi * GGML_CPU_FP16_TO_FP32(x[lb].d) * GGML_CPU_FP16_TO_FP32(y[lb].d);
        }
    }
}
```

图 4: q4_0 固定 vl = qk / 2 和 m1

```
void ggml_vec_dot_q4_0_q8_0(int n, float * GGML_RESTRICT s, size_t bs, const void * GGML_RESTRICT vx, size_t bx, const void * GGML_RESTRICT vy, size_t by, int nrc) {
    #if defined(__riscv_v)
        const int qk = QK8_0;
        const int nb = n / qk;

        assert(n % qk == 0);
        assert(nrc == 1);
        UNUSED(nrc);
        UNUSED(bx);
        UNUSED(by);
        UNUSED(bs);

        const block_q4_0 * GGML_RESTRICT x = vx;
        const block_q8_0 * GGML_RESTRICT y = vy;

        int lb = 0;
        float sumf = 0;

        size_t vl = qk / 2;

        for (; lb < nb; ++lb) {
            // load elements
            vuint8m1_t tx = __riscv_vle8_v_u8m1(x[lb].qs, vl);

            vint8m1_t y0 = __riscv_vle8_v_i8m1(y[lb].qs, vl);
            vint8m1_t y1 = __riscv_vle8_v_i8m1(y[lb].qs+16, vl);

            // mask and store lower part of x, and then upper part
            vuint8m1_t x_a = __riscv_vand_vx_u8m1(tx, 0x0F, vl);
            vuint8m1_t x_l = __riscv_vsrl_vx_u8m1(tx, 0x04, vl);

            vint8m1_t x_ai = __riscv_vreinterpret_v_u8m1_i8m1(x_a);
            vint8m1_t x_li = __riscv_vreinterpret_v_u8m1_i8m1(x_l);

            // subtract offset
            vint8m1_t v0 = __riscv_vsub_vx_i8m1(x_ai, 8, vl);
            vint8m1_t v1 = __riscv_vsub_vx_i8m1(x_li, 8, vl);

            vint16m2_t vec_mul1 = __riscv_vmmul_vv_i16m2(v0, y0, vl);
            vint16m2_t vec_mul2 = __riscv_vmmacc_vv_i16m2(vec_mul1, v1, y1, vl);

            vint32m1_t vec_zero = __riscv_vmv_v_x_i32m1(0, vl);
            vint32m1_t vs2 = __riscv_vwredsum_vs_i16m2_i32m1(vec_mul2, vec_zero, vl);

            int sumi = __riscv_vmv_v_s_i32m1_i32(vs2);

            sumf += sumi * GGML_CPU_FP16_TO_FP32(x[lb].d) * GGML_CPU_FP16_TO_FP32(y[lb].d);
        }
    }
}
```

图 5: q5_K 固定 vl = 8、vl = 32

```

void ggml_vec_dot_q4_0_q8_0(int n, float * GGML_RESTRICT s, size_t bs, const void * GGML_RESTRICT vx, size_t bx, const void * GGML_RESTRICT vy, size_t by, int nrc) {
    #if defined(__riscv_v)
        const int qk = QK8_0;
        const int nb = n / qk;

        assert(n % qk == 0);
        assert(nrc == 1);
        UNUSED(nrc);
        UNUSED(bx);
        UNUSED(by);
        UNUSED(bs);

        const block_q4_0 * GGML_RESTRICT x = vx;
        const block_q8_0 * GGML_RESTRICT y = vy;

        int lb = 0;
        float sumf = 0;

        size_t vl = qk / 2;

        for (; lb < nb; ++lb) {
            // load elements
            vint8m1_t tx = __riscv_vle8_v_u8m1(x[lb].qs, vl);

            vint8m1_t y0 = __riscv_vle8_v_i8m1(y[lb].qs, vl);
            vint8m1_t y1 = __riscv_vle8_v_i8m1(y[lb].qs+16, vl);

            // mask and store lower part of x, and then upper part
            vint8m1_t x_a = __riscv_vand_vx_u8m1(tx, 0x0f, vl);
            vint8m1_t x_l = __riscv_vsr1_vx_u8m1(tx, 0x04, vl);

            vint8m1_t x_ai = __riscv_vreinterpret_v_u8m1_i8m1(x_a);
            vint8m1_t x_li = __riscv_vreinterpret_v_u8m1_i8m1(x_l);

            // subtract offset
            vint8m1_t v0 = __riscv_vsub_vx_i8m1(x_ai, 8, vl);
            vint8m1_t v1 = __riscv_vsub_vx_i8m1(x_li, 8, vl);

            vint16m2_t vec_mul1 = __riscv_vmul_vv_i16m2(v0, y0, vl);
            vint16m2_t vec_mul2 = __riscv_vmmacc_vv_i16m2(vec_mul1, v1, y1, vl);

            vint32m1_t vec_zero = __riscv_vmv_v_x_i32m1(0, vl);
            vint32m1_t vs2 = __riscv_vwredsum_vs_i16m2_i32m1(vec_mul2, vec_zero, vl);

            int sumi = __riscv_vmv_v_s_i32m1_i32(vs2);

            sumf += sumi * GGML_CPU_FP16_TO_FP32(x[lb].d) * GGML_CPU_FP16_TO_FP32(y[lb].d);
        }
    #endif
}

```

图 6: MXFP4 只有 vl128 和 vl256 路径

3 项目实现方案

3.1 为 SSM_SCAN 增加 RVV 向量化路径

3.1.1 实施目标

针对 ggml/src/ggml-cpu/ops.cpp:9634-9870 的 GGML_OP_SSM_SCAN，优先实现 Mamba-2 路径的 RVV kernel，使状态更新和输出归约沿 d_state 维度使用 VLA 向量指令。

当前 RVV 分支位于 ops.cpp:9738-9740:

```

#elif defined(__riscv_v_intrinsic)
    // todo: RVV implementation
    const int np = 0;

```

由于 np 为 0，后续计算全部进入 ops.cpp:9777-9785 的标量剩余循环。

Mamba-2 的核心循环位于 ops.cpp:9692-9777，计算关系为：

```

dt_soft_plus = softplus(dt[h])
dA           = exp(dt_soft_plus * A[h])
x_dt         = x[ii] * dt_soft_plus

state[k] = previous_state[k] * dA + B[k] * x_dt
output   = sum(state[k] * C[k])

```

不能沿 token 维度并行化，应该沿 d_state 维度进行 RVV 向量化。

3.1.2 实现

在 ops.cpp:9738 的 RVV 分支中增加向量循环：

```

for (k = 0; k < d_state; ) {
    vl = vsetvl(d_state - k)

    load previous_state[k : k+vl]
    load B[k : k+vl]
    load C[k : k+vl]

    state = previous_state * dA + B * x_dt
    acc   = acc + state * C

    store state[k : k+vl]
    k += vl
}

```

3.2 为 Gated-Delta-Net 增加算子级 RVV 优化路径

3.2.1 实施目标

针对 `ggml/src/ggml-cpu/ops.cpp:10752-10903` 中的 `GGML_OP_GATED_DELTA_NET`，优化状态矩阵更新、delta 计算和 attention 输出计算，使其沿 S_v 维度使用 RVV 向量指令。

当前核心计算位于 `ops.cpp:10859-10888`，主要包括：

```

state = state * exp(g)
delta[j] = dot(state[j], k)
state[j] += delta[j] * k
output[j] = dot(state[j], q)

```

当前实现通过多层循环调用通用 `ggml_vec_mul_f32`、`ggml_vec_dot_f32` 和 `ggml_vec_mad_f32`，尚未形成面向 Gated-Delta-Net 的专用 RVV kernel。

此外，SpacemiT backend 中的算子路由仍未启用，位置为 `ggml/src/ggml-cpu/spacemit/ime.cpp:1142-1144`：

```

// TODO For GGML_OP_GATED_DELTA_NET
// case GGML_OP_GATED_DELTA_NET:

```

3.2.2 实现

保持现有状态矩阵布局不变。当前状态矩阵以转置形式存储：

```

s_out[j * S_v + i] = state[i][j]

```

因此可以沿连续的 S_v 维度进行向量化。针对每个 head 和 sequence，使用如下 RVV 计算流程：

```

for (j = 0; j < S_v; ++j) {
    vl = vsetvl(S_v - i)

    load state[j][i : i + vl]
    load k[i : i + vl]
    load q[i : i + vl]

    delta[j] = dot(state[j], k)
}

```

```

    state[j] = state[j] + delta[j] * k
    output[j] = dot(state[j], q)

    store state[j][i : i + vl]
}

```

对于 KDA 模式，保留现有的逐元素 gate 语义：

```

gate[i] = exp(g[i])
state[j][i] = state[j][i] * gate[i]

```

其中 gate 的指数计算可以暂时使用标量 `expf`，后续使用 RVV 向量指数实现进行优化。状态更新、点积和外积更新使用 RVV F32 load、store、FMA 和 reduction 指令。

实现过程中保持以下行为不变：

- token 维度仍按顺序执行，保证 recurrent state 的依赖关系；
- head 和 sequence 维度继续复用当前线程划分；
- 保留 $K > 1$ 时的 state snapshot 和 rollback 逻辑；
- 对不满足数据类型或 shape 条件的输入回退到当前通用 F32 实现。

在 `spacemit/ime.cpp:1142-1144` 增加 Gated-Delta-Net dispatch，并根据 S_v 、数据类型和 VLEN 条件选择 SpacemiT RVV kernel。对于不满足条件的输入，继续调用通用 CPU 路径。

3.3 面向 VLEN=1024 的量化点积内核优化

3.3.1 实施目标

针对高频量化类型 `q4_0`、`q8_0` 和 `q5_K`，优化 `ggml/src/ggml-cpu/arch/riscv/quants.c` 中的 RVV 点积路径，充分利用 VLEN=1024 处理器的向量寄存器宽度。

当前相关实现位置为：

- `q4_0`: `quants.c:222-273`;
- `q8_0`: `quants.c:435-479`;
- `q5_K`: `quants.c:2081-2197`;
- `MXFP4`: `quants.c:6469-6600`。

现有代码中存在固定 VL 或有限 VLEN 分发。例如 `q5_K` 路径中使用：

```

vl = 8;
...
vl = 32;

```

MXFP4 路径目前只区分 VL128 和 VL256：

```

ggml_vec_dot_mxfp4_q8_0_vl128(...)
ggml_vec_dot_mxfp4_q8_0_vl256(...)

```

这些路径没有针对 VLEN=1024 统一选择最优的 VL、LMUL 和 block 处理粒度。

3.3.2 实现

根据量化 block 的数据布局，重新设计 RVV 点积的向量处理粒度。基本流程如下：

```
detect vlen = __riscv_vlenb() * 8

select LMUL and vector type
select vl = vsetvl(element_count)

for each quantization block {
    load packed quantized values
    unpack low and high nibbles
    load q8 activation values

    widen multiply quantized values and activations
    accumulate partial products
    reduce vector accumulator

    apply block scale and zero-point
}
```

针对 VLEN=1024 平台，重点评估以下策略：

- 使用 LMUL=1、LMUL=2 和 LMUL=4 的寄存器压力；
- 一次处理一个或多个量化 block；
- q4/q5 unpack 与 widening multiply 的指令开销；
- 多个累加器对向量归约延迟的隐藏效果；
- VLEN=128、256 和 1024 下的统一 fallback 行为。

4 项目计划

本项目按照“基线建立、算子实现、性能调优、测试验证和上游提交”的顺序推进。优先完成 SSM_SCAN，再完成 Gated-Delta-Net 和量化点积优化，最后进行统一性能评估和 PR 整理。

4.1 第一阶段：代码分析与基线建立（第 1-2 周）

- 阅读 ops.cpp、quants.c、ime.cpp 和现有 RVV kernel；
- 在 K3 VLEN=1024 平台完成 llama.cpp 编译和运行环境配置；
- 使用 test-backend-ops 验证 SSM_SCAN、Gated-Delta-Net 和量化算子的当前正确性；
- 使用 llama-bench 建立算子级和模型级 baseline；
- 记录不同 d_state、 S_v 、token 数量和量化类型下的执行时间。

阶段产出：

- K3 平台构建说明；
- 算子级 baseline 数据；
- 模型级 baseline 数据；
- 初步性能瓶颈分析。

4.2 第二阶段：实现 SSM_SCAN 的 RVV 路径（第 3–5 周）

- 在 ops.cpp:9738 的 RVV 分支中实现 Mamba-2 状态更新；
- 沿 d_state 维度使用 vsetv1、F32 load/store、FMA 和 reduction；
- 处理非完整向量长度的 tail；
- 保持现有多线程、multi-sequence 和 rollback snapshot 逻辑；
- 复用现有 SSM_SCAN 测试验证 Mamba-1、Mamba-2 和 rollback 场景；
- 在 K3 上比较 RVV 路径与 baseline 的性能。

阶段产出：

- SSM_SCAN RVV 实现；
- correctness 测试结果；
- 算子级性能数据；
- 第一个 focused PR。

4.3 第三阶段：实现 Gated-Delta-Net RVV 优化路径（第 6–8 周）

- 优化状态矩阵衰减、delta 计算、外积更新和 attention 输出；
- 沿连续的 S_o 维度进行 RVV 向量化；
- 保持 KDA 和 scalar-gate 两种计算模式；
- 保留 state snapshot、rollback 和现有线程调度逻辑；
- 在 spacemit/ime.cpp 中增加满足条件时的 Gated-Delta-Net dispatch；
- 对不满足数据类型或 shape 条件的输入保留通用 CPU fallback；
- 复用现有 Gated-Delta-Net 测试验证不同 head、token 和 state 配置。

阶段产出：

- Gated-Delta-Net RVV 优化实现；
- SpacemiT backend 路由；
- correctness 和性能测试结果；
- 第二个 focused PR。

4.4 第四阶段：优化 VLEN=1024 量化点积（第 9–10 周）

- 对 q4_0、q8_0 和 q5_K 进行指令级 profiling；
- 比较不同 LMUL、VL 和循环展开策略；
- 优化量化数据 unpack、widening multiply 和向量归约；
- 针对 VLEN=1024 选择合适的 block 处理粒度；
- 保持 VLEN=128、256 以及非 RVV 平台的 fallback 行为；
- 如果 MXFP4 的数据布局和 repack kernel 验证通过，则补充 MXFP4 dispatch。

阶段产出：

- q4_0、q8_0 或 q5_K 的 VLEN=1024 优化；
- 不同 LMUL 和 tile 配置的对比数据；
- 第三个 focused PR。

4.5 第五阶段：综合测试与性能评估（第 11 周）

- 运行 test-backend-ops 和现有 RISC-V CI；
- 对比优化前后的算子输出和 state snapshot；
- 使用 llama-bench 测试 prompt processing 和 generation；
- 在相同模型、线程数、batch size 和编译参数下重复测试；
- 计算各 workload 的加速比和几何平均加速比；
- 分析优化对端到端模型性能的影响。

验收指标：

- 优化算子通过现有 correctness 测试；
- 非 RVV 平台和不满足条件的输入保持原有 fallback 行为；
- K3 VLEN=1024 平台选定 workload 的几何平均加速比达到 $1.10\times$ 以上；
- 至少完成三个独立的 upstream PR。

4.6 第六阶段：代码整理与上游协作（第 12 周）

- 根据 benchmark 结果清理无收益或不必要的代码；
- 补充必要的代码注释、构建说明和性能数据；
- 将 SSM_SCAN、Gated-Delta-Net 和量化点积优化保持为独立 PR；
- 根据 mentor 和社区 review 意见进行修改；
- 整理最终技术报告，记录实现方法、测试环境、性能结果和已知限制。

最终预期提交不少于三个 focused PR，并根据上游 review 结果持续迭代，争取至少两个 PR 被 llama.cpp 上游合并。

5 参考文献

1. RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Chapter 30: Vector Extension, Version 1.0*. 2021. <https://github.com/riscv/riscv-v-spec>
2. Gerganov, G., et al. *llama.cpp*. GitHub repository. <https://github.com/ggml-org/llama.cpp>
3. Gerganov, G., et al. *GGML Tensor Library*. Source code in the llama.cpp repository. <https://github.com/ggml-org/llama.cpp/tree/master/ggml>
4. Gu, A., and Dao, T. *Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. Proceedings of the 40th International Conference on Machine Learning, 2024. <https://arxiv.org/abs/2405.21060>
5. Yang, Y., et al. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. arXiv preprint arXiv:2412.06464, 2024. <https://arxiv.org/abs/2412.06464>
6. llama.cpp Contributors. *RISC-V RVV Quantization and Matrix Multiplication Kernels*. llama.cpp source files: `ggml/src/ggml-cpu/arch/riscv/quants.c` and `ggml/src/ggml-cpu/arch/riscv/repack.cpp`. <https://github.com/ggml-org/llama.cpp>
7. llama.cpp Contributors. *llama-bench Benchmark Tool*. llama.cpp documentation. <https://github.com/ggml-org/llama.cpp/tree/master/tools/llama-bench>